

Voiceover Script — Read Aloud While Recording

Project: Auditing and Repairing a Published Nepali ASR Model **Total runtime target:** ~7–9 minutes | 8 slides

Read each block on its matching slide. Pause briefly between slides.

SLIDE 1 — Title (~40 sec)

Hi, I'm Bimochan Raj Kunwar. This is my ST7088CEM Artificial Neural Networks project: auditing a published Nepali speech recognition model, and then repairing it.

There's a public Nepali ASR model on Hugging Face that claims a 5.97% word error rate — remarkably good for a low-resource language. I wanted to find out: does that number actually hold up on real, diverse speech?

Spoiler — it doesn't, by a lot. Let me show you.

SLIDE 2 — The Problem (~65 sec)

This model was benchmarked on OpenSLR-43 — a dataset recorded by a single female speaker.

That's the problem: a benchmark on one voice tells you almost nothing about how the model handles different speakers, accents, or recording conditions. This is a common issue in low-resource ASR research generally, not just this one model.

So I framed two research questions. RQ1: does the published claim survive contact with genuinely diverse speech? RQ2: if not, can I repair the model by fine-tuning it, and is any improvement genuine or just a measurement artefact?

SLIDE 3 — Architecture (~90 sec)

Here's the model architecture — XLS-R, a wav2vec2 variant.

Raw audio goes through a convolutional feature encoder — seven conv layers that turn the waveform into a sequence of latent frames. That feeds a 24-layer transformer encoder, which builds contextualised representations using self-attention. Finally a linear layer plus a CTC head produces per-frame character probabilities, decoded into Devanagari text.

The important part for this project: I kept the CNN feature encoder frozen the whole time — in both the zero-shot audit and the fine-tuning. When I fine-tune, I only update the transformer encoder and the CTC head, using the CTC loss.

I also keep the checkpoint's own vocabulary — I'm repairing this exact model, not training a new one or swapping in a different architecture like Whisper, which I actually considered and dropped early on, because changing architecture would confound the comparison.

SLIDE 4 — Task 1: The Audit (~45 sec)

Task 1: I reproduced the published number in-domain and got 4.91% WER — actually slightly better, so the claim technically holds on its own data.

Then I ran the exact same checkpoint, zero-shot, on OpenSLR-54 — a completely different, multi-speaker Nepali corpus, 160 speakers. WER jumped to 62.30%. That's a twelve-fold degradation.

So RQ1's answer is no — the published number does not describe real-world, multi-speaker performance.

SLIDE 5 — Task 2: The Repair (~45 sec)

Task 2: I fine-tuned that same checkpoint on a 15-hour, 160-speaker, speaker-disjoint subset of OpenSLR-54 — meaning zero speaker overlap between train, validation, and test.

Out-of-domain WER fell from 62.30% to 38.17% — a 39% relative improvement.

I'm also honest about the cost: in-domain WER rose from 4.91% to 16.40%, because the model is now adapted toward diverse speech rather than that one original speaker. That's an expected specialisation trade-off, not a bug.

SLIDE 6 — The Ablation (~75 sec)

Here's the part I think matters most methodologically.

I trained a second model, identical setup, identical budget — but on a leaky split, where each speaker's utterances are scattered randomly across train, validation, and test instead of being kept separate.

That leaky model scored 32.55% — better than the properly disjoint model's 38.17%. But that's not a real improvement — it's leakage. The leaky test set isn't truly unseen, because the model has already partially learned those speakers' voices during training.

This confirms that if I — or anyone else — evaluates without controlling for speaker overlap, the reported number is inflated. It's the same failure mode that produced the original misleading 5.97% benchmark, just demonstrated at a smaller scale on my own data.

SLIDE 7 — Efficiency (~50 sec)

Last piece: I also benchmarked deployment efficiency.

Dynamic INT8 quantization shrank the model by 96% — from about 1.2 gigabytes to 48.5 megabytes. But it was actually 1.64 times slower on this Apple Silicon hardware, not faster, because the fast quantized kernels aren't available outside x86.

I'm reporting that honestly rather than only showing the size win — a deployment optimisation that looks good on paper doesn't automatically transfer to every platform.

SLIDE 8 — Conclusion (~40 sec)

So: RQ1 — the published 5.97% claim does not generalise to diverse speakers, degrading to 62.30% WER.

RQ2 — yes, fine-tuning genuinely repairs a large share of that gap, down to 38.17%, and the speaker-leakage ablation confirms that improvement is real, not a measurement artefact.

Everything here — code, MLflow experiment logs, data manifests, and the full report — is public on GitHub, link on screen and in the description.

Thanks for watching.

Total read-aloud time at a natural pace: roughly 7.5–8 minutes. If you need it shorter, the safest cuts are trimming Slide 3's Whisper aside and Slide 2's "common issue in low-resource ASR" sentence — both are context, not core findings.